

Syllabus

UNIT IV

Query Optimization: Introduction, steps of optimization, various algorithms to implement select, project and join operations of relational algebra, optimization methods: heuristic based, cost estimation based.

Transaction Processing, Concurrency Control and Recovery Management: Transaction Model properties, State Serializability, Lock base protocols, Two Phase Locking, Time Stamping Protocols for Concurrency Control, and Validation Based Protocol, Multiple Granularities, Granularity of Data Item. Multi version schemes, Recovery with Concurrent Transaction, Recovery technique based on Deferred Update and Immediate Update, Shadow Paging, Recovery in Multi Database System and Database Backup and Recovery from Catastrophic Failure

Query Optimization

- **Query:** A query is a request for information from a database.

Query Processing:

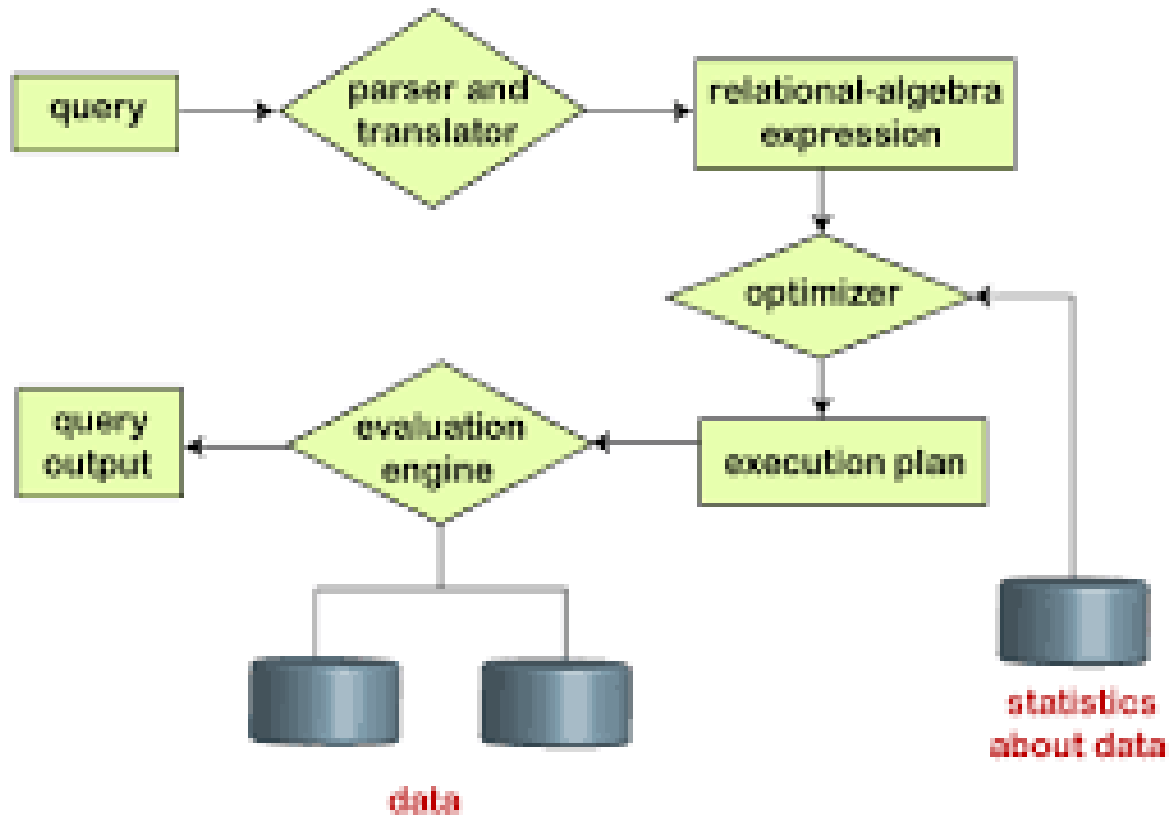
- Query Processing is a translation of high-level queries into low-level expression.
- It is a step wise process that can be used at the physical level of the file system, query optimization and actual execution of the query to get the result.
- It requires the basic concepts of relational algebra and file structure.
- It refers to the range of activities that are involved in extracting data from the database.
- It includes translation of queries in high-level database languages into expressions that can be implemented at the physical level of the file system.

Query Optimization:

- A single query can be executed through different algorithms or re-written in different forms and structures.
- Hence, the question of query optimization comes into the picture – Which of these forms or pathways is the most optimal? The query optimizer attempts to determine the most efficient way to execute a given query by considering the possible query plans.

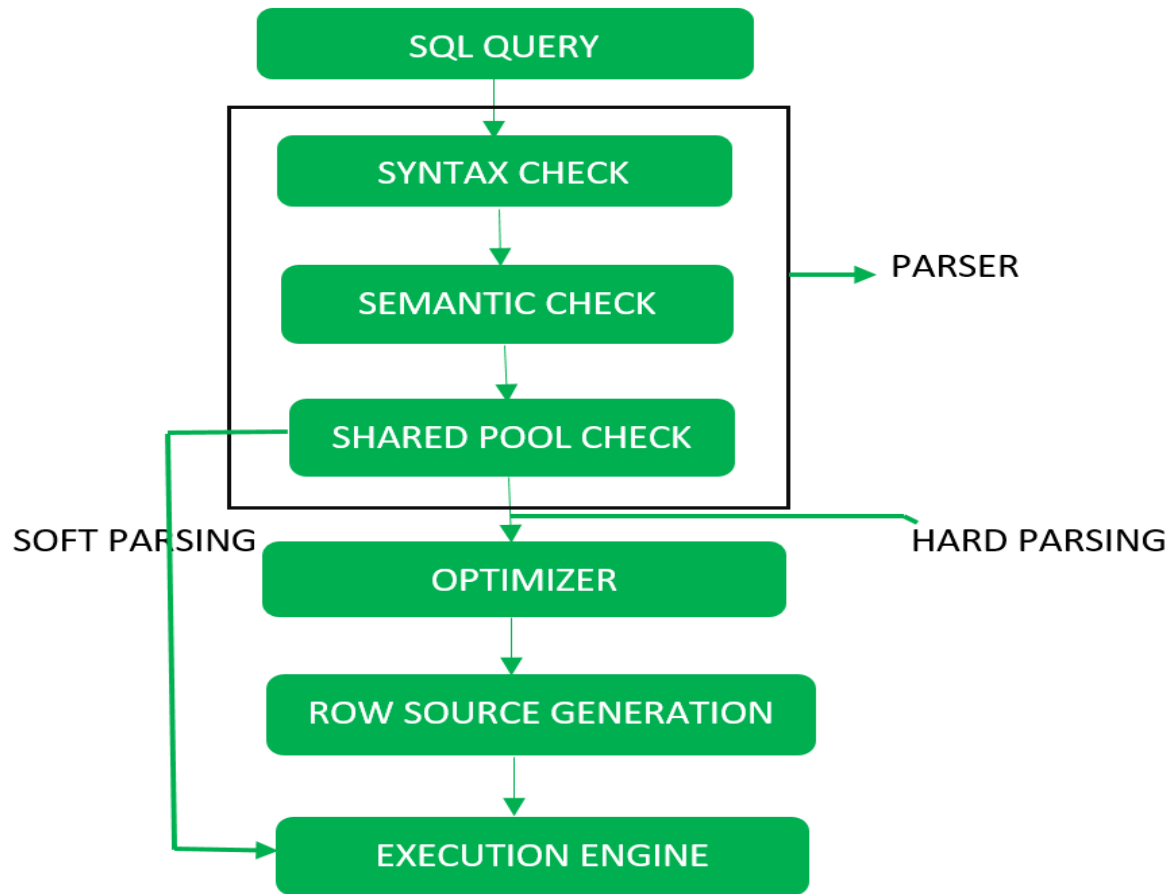
Importance: The goal of query optimization is to reduce the system resources required to fulfill a query, and ultimately provide the user with the correct result set faster.

- In query processing, we will understand how these queries are processed and how they are optimized.



Steps in query processing

- Detailed Diagram is drawn as:.



In the above diagram,

1. Parser & Translator:- The first step is to transform the query into a standard form.

- A query is translated into SQL and into a relational algebraic expression.
- During this process, **Parser perform** the syntax check , Semantic check and Shared pool check and verifies the relations and the attributes which are used in the query.
- Parser performs the following checks as (refer detailed diagram):

A. Syntax check – concludes SQL syntactic validity.

Example: SELECT * **FORM** employee

- Here error of wrong spelling of FROM is given by this check.

1. Parser & Translator:-

B. Semantic check – determines whether the statement is meaningful or not. Example: query contains a tablename which does not exist is checked by this check.

C.Shared Pool check – Every query possess a hash code during its execution. So, this check determines existence of written hash code in shared pool if code exists in shared pool then database will not take additional steps for optimization and execution.

2. The second step is **Query Optimizer**. In this, it transforms the query into equivalent expressions that are more efficient to execute.

- During optimization stage, database must perform a hard parse atleast for one unique DML statement and perform optimization during this parse. This database never optimizes DDL unless it includes a DML component such as subquery that require optimization.

- It is a process in which multiple query execution plan for satisfying a query are examined and most efficient query plan is satisfied for execution.

- Database catalog stores the execution plans and then optimizer passes the lowest cost plan for execution.

3.The third step is **Query evaluation**. It executes the above query execution plan and returns the result.

Translating SQL Queries into Relational Algebra

Example: Fetch the details of 1st semester students

Studentinfo

ID	NAME	SEM
1	A	5
2	B	6
3	C	1
4	D	1

SQL Query:

Select*from studentinfo where sem=1;

Relational Algebra:

Π : Projection

σ : Selection

$\Pi_{ID, name, sem} (\sigma_{sem=1}(studentinfo))$

Example:

Query:

SELECT Ename FROM Employee WHERE Salary > 5000;

Translated into Relational Algebra Expression

$\sigma_{\text{Salary} > 5000} (\pi_{\text{Ename}} (\text{Employee}))$

OR

$\pi_{\text{Ename}} (\sigma_{\text{Salary} > 5000} (\text{Employee}))$

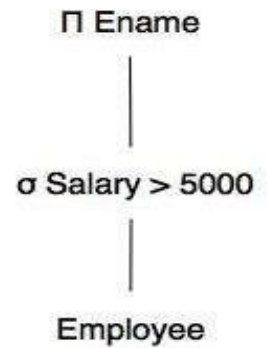


Fig. Query Execution Plan

- A sequence of primitive operations that can be used to evaluate a query is a Query Execution Plan or Query Evaluation Plan.
- The above diagram indicates that the query execution engine takes a query execution plan and returns the answers to the query.
- Query Execution Plan minimizes the cost of query evaluation.

Methods of Query Optimization

There are two methods of query optimization

- Cost based Optimization (Physical)
- Heuristic Optimization (Logical)

1. Cost based Optimization (Physical):

- This is based on the cost of the query. The query can use different paths based on indexes, constraints, sorting methods etc.
- This method mainly uses the statistics like record size, number of records, number of records per block, number of blocks, table size, whether whole table fits in a block, organization of tables, uniqueness of column values, size of columns etc.
- Assign cost to Different strategies then it estimate and compare their cost of different queries.

Methods of Query Optimization....

Cost based Optimization (Physical):

The cost of executing a query includes...

1. **Cost of accessing secondary storage:** cost of searching, reading and writing disk blocks
2. **Storage cost:** Cost of storing intermediate results
3. **Computation Cost:** Cost of performing in memory operation
4. **Memory usage Cost:** Cost related to number of occupied memory buffers.
5. **Communication cost:** Cost of communicating the query result from one side to another.

Methods of Query Optimization..

Cost based Optimization (Physical):

Problems:

1. Cost function required to assign cost to each
2. Number of execution strategies to be considered not fixed.
3. Sometime be time being to compare cost
4. Does not guarantee finding best strategies
5. Not time effective to consider every possible strategies
6. It is time being and expensive technique

2. Heuristic Optimization (Logical):

- Optimizer apply heuristics rules to find the optimized query execution strategies. Rules are called heuristics, Usually it helps to reduce the cost of execution of query.
- This method is also known as rule-based optimization.
- This is based on the equivalence rule on relational expressions; hence the number of combination of queries get reduces here. Hence the cost of the query too reduces.
- It is based on query tree
- This method creates relational tree for the given query based on the equivalence rules. These equivalence rules by providing an alternative way of writing and evaluating the query, gives the better path to evaluate the query. This rule need not be true in all cases. It needs to be examined after applying those rules. The most important set of rules followed in this method is listed below:

Heuristic Optimization (Logical):

- Perform all the selection operation as early as possible in the query. This should be first and foremost set of actions on the tables in the query. By performing the selection operation, we can reduce the number of records involved in the query, rather than using the whole tables throughout the query.

Heuristic Rules:

- It used as an optimized technique to modify the internal representation of the queries.
- Uses set of rules that typically improve execution performance of input query.
- **1. Apply select operation** before project operation then join or binary operations are used
- **2. Apply Project operation** before join or other binary operations.
- Select and project reduce the size of file hence applied before join . Or binary.
- Size of resulting relation form a binary operation like join is very large.

Transaction

- A transaction is an action or series of actions that are being performed by a single user or application program, which reads or updates the contents of the database.
- A transaction can be defined as a logical unit of work on the database. This may be an entire program, a piece of a program, or a single command (like the SQL commands such as INSERT or UPDATE), and it may engage in any number of operations on the database.
- Let's take an example of a simple transaction. Suppose a bank employee transfers Rs 500 from A's account to B's account. This very simple and small transaction involves several low-level tasks.

A's Account

Open_Account(A)

Old_Balance = A.balance

New_Balance = Old_Balance - 500

A.balance = New_Balance

Close_Account(A)

B's Account

Open_Account(B)

Old_Balance = B.balance

New_Balance = Old_Balance + 500

B.balance = New_Balance

Close_Account(B)

Operations of Transaction:

Following are the main operations of transaction:

Read(X): Read operation is used to read the value of X from the database and stores it in a buffer in main memory.

Write(X): Write operation is used to write the value back to the database from the buffer.

Let's take an example to debit transaction from an account which consists of following operations:

1. $R(X);$
2. $X = X - 500;$
3. $W(X);$

Let's assume the value of X before starting of the transaction is 4000. The first operation reads X's value from database and stores it in a buffer. The second operation will decrease the value of X by 500. So buffer will contain 3500. The third operation will write the buffer's value to the database. So X's final value will be 3500. But it may be possible that because of the failure of hardware, software or power, etc. that transaction may fail before finished all the operations in the set.

For example: If in the above transaction, the debit transaction fails after executing operation 2 then X's value will remain 4000 in the database which is not acceptable by the bank.

To solve this problem, we have two important operations:

Commit: It is used to save the work done permanently.

Rollback: It is used to undo the work done.

Properties of Transaction (ACID PROPERTIES):

- The transaction has the four properties. These are used to maintain consistency in a database, before and after the transaction.
- A transaction in a database system must maintain **Atomicity, Consistency, Isolation, and Durability** commonly known as ACID properties in order to ensure accuracy, completeness, and data integrity.

Atomicity

means either all successful or none.

Consistency

ensures bringing the database from one consistent state to another consistent state.
ensures bringing the database from one consistent state to another consistent state.

Isolation

ensures that transaction is isolated from other transaction.

Durability

means once a transaction has been committed, it will remain so, even in the event of errors, power loss etc.

1. Atomicity

It states that all operations of the transaction take place at once if not, the transaction is aborted. There is no midway, i.e., the transaction cannot occur partially. Each transaction is treated as one unit and either run to completion or is not executed at all.

Atomicity involves the following two operations:

Abort: If a transaction aborts then all the changes made are not visible.

Commit: If a transaction commits then all the changes made are visible.

Example: Let's assume that following transaction T consisting of T1 and T2. A consists of Rs 600 and B consists of Rs 300. Transfer Rs 100 from account A to account B.

T1	T2
Read(A)	Read(B)
A:= A-100	Y:= Y+100
Write(A)	Write(B)

After completion of the transaction, A consists of Rs 500 and B consists of Rs 400. If the transaction T fails after the completion of transaction T1 but before completion of transaction T2, then the amount will be deducted from A but not added to B. This shows the inconsistent database state. In order to ensure correctness of database state, the transaction must be executed in entirety.

2. Consistency – The database must remain in a consistent state after any transaction. No transaction should have any adverse effect on the data residing in the database. If the database was in a consistent state before the execution of a transaction, it must remain consistent after the execution of the transaction as well.

Referring to the example above,

The total amount before and after the transaction must be maintained.

For example: The total amount must be maintained before or after the transaction.

Total **before T** occurs = $500 + 200 = 700$.

Total **after T** occurs = $400 + 300 = 700$.

Therefore, database is **consistent**. Inconsistency occurs in case **T1** completes but **T2** fails. As a result T is incomplete.

3. Durability

The durability property is used to indicate the performance of the database's consistent state. It states that the transaction made the permanent changes.

They cannot be lost by the erroneous operation of a faulty transaction or by the system failure. When a transaction is completed, then the database reaches a state known as the consistent state. That consistent state cannot be lost, even in the event of a system's failure.

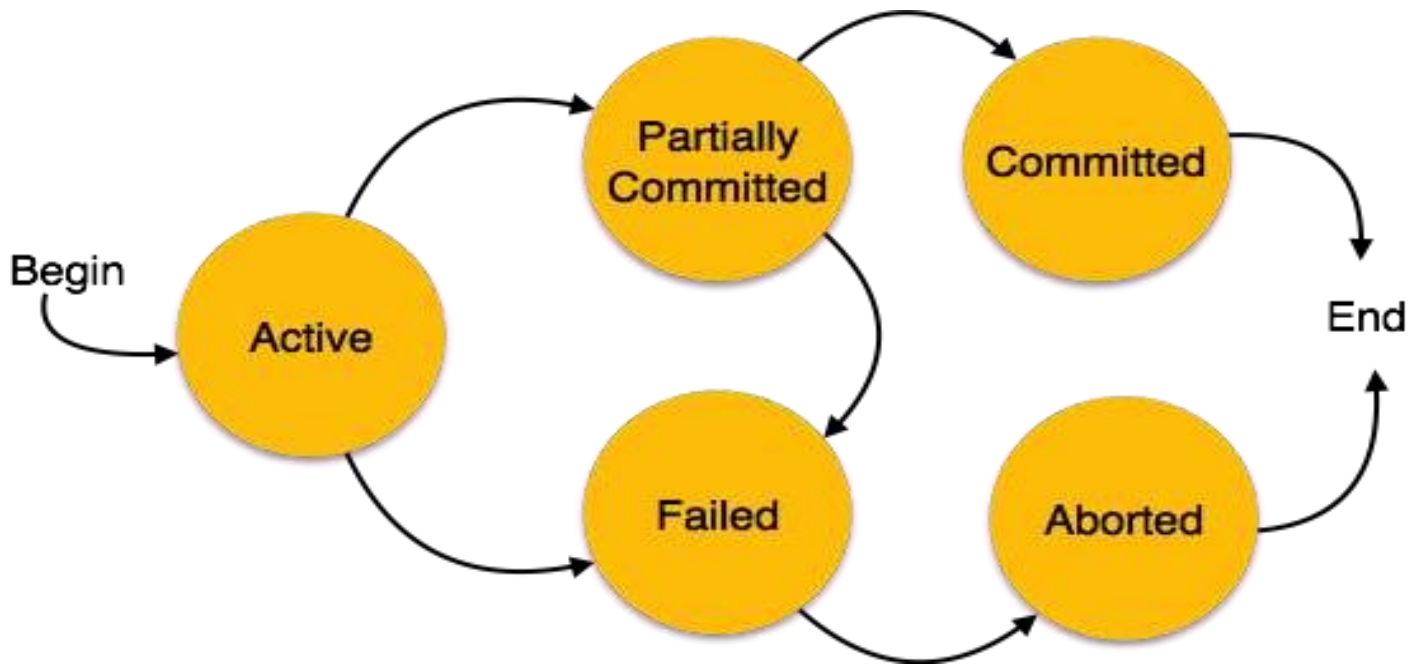
The recovery subsystem of the DBMS has the responsibility of Durability property.

4. Isolation –

- In a database system where more than one transaction are being executed simultaneously and in parallel, the property of isolation states that all the transactions will be carried out and executed as if it is the only transaction in the system. No transaction will affect the existence of any other transaction.

States of Transactions

A transaction in a database can be in one of the following states –



- **Active** – In this state, the transaction is being executed. This is the initial state of every transaction.
- For example: Insertion or deletion or updating a record is done here. But all the records are still not saved to the database.
- **Partially Committed** – When a transaction executes its final operation, it is said to be in a partially committed state. but the data is still not saved to the database.
- **Failed** – A transaction is said to be in a failed state if any of the checks made by the database recovery system fails. A failed transaction can no longer proceed further.

- **Aborted** – If any of the checks fails and the transaction has reached a failed state, then the recovery manager rolls back all its write operations on the database to bring the database back to its original state where it was prior to the execution of the transaction. Transactions in this state are called aborted. The database recovery module can select one of the two operations after a transaction aborts –

Re-start the transaction

Kill the transaction

If the transaction fails in the middle of the transaction then before executing the transaction, all the executed transactions are rolled back to its consistent state.

- **Committed** – If a transaction executes all its operations successfully, it is said to be committed. All its effects are now permanently established on the database system.

Schedule

- This schedule determines the exact order of operations that are going to be performed on database. **A series of operation from one transaction to another transaction is known as schedule.** It is used to preserve the order of the operation in each of the individual transaction.
- It is a bundle of transaction executing together.
- When transactions are executing concurrently in an interleaved fashion, then the order of execution of operation from the various transaction is known as schedule.
- Transaction processing system usually allow multiple transaction to run concurrently.

For example:

- Suppose T1 & T2 be two transactions that transfer funds from one account to another. Transaction T1 transfer Rs50 from account A to account B. It is defined in in Fig 16.4
- Transaction T2 Transfers 10 percent of the balance from account A to account B. It is defined in fig 16.5

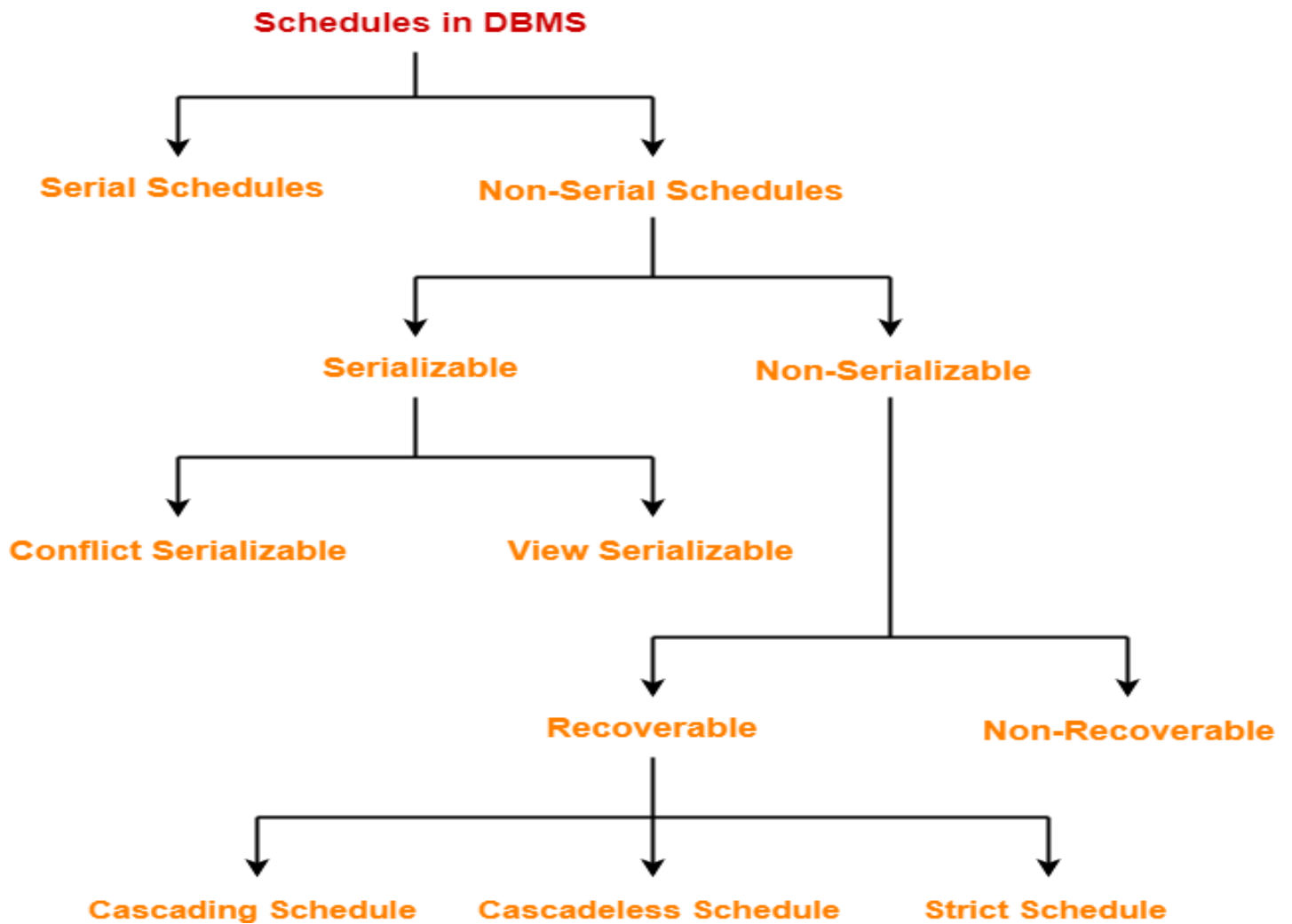
```
T1 : read (A);  
      A := A-50;  
      write (A);  
      read (B);  
      B := B + 50;  
      write (B);
```

Fig. 16.4 Transfers from A to B

```
T2 : read (A)  
      temp := A * 0.1;  
      A := A-temp;  
      write (A);  
      read (B);  
      B := B + temp;  
      write (B);
```

Fig. 16.5 Transfers 10 percent of the balance from account A to B

Types of Schedules



Types of Schedule

1. Serial Schedule:

- In **Serial schedule**, a transaction is executed completely before starting the execution of another transaction.
- In other words, you can say that in serial schedule, a transaction does not start execution until the currently running transaction finished execution.
- This type of execution of transaction is also known as **non-interleaved** execution. The example we have seen above is the serial schedule.

Transaction T1	Transaction T2
R (A)	
W (A)	
R (B)	
W (B)	
Commit	
	R (A)
	W (B)
	Commit

In this schedule,

- There are two transactions T1 and T2 executing serially one after the other.
- Transaction T1 executes first.
- After T1 completes its execution, transaction T2 executes.

So, this schedule is an example of a Serial Schedule.

Another Example:

In this schedule,

- There are two transactions T1 and T2 executing serially one after the other.
- Transaction T2 executes first.
- After T2 completes its execution, transaction T1 executes.

So, this schedule is an example of a **Serial Schedule**.

Transaction T1	Transaction T2
	R (A)
	W (B)
	Commit
R (A)	
W (A)	
R (B)	
W (B)	
Commit	

Types of Serializability...

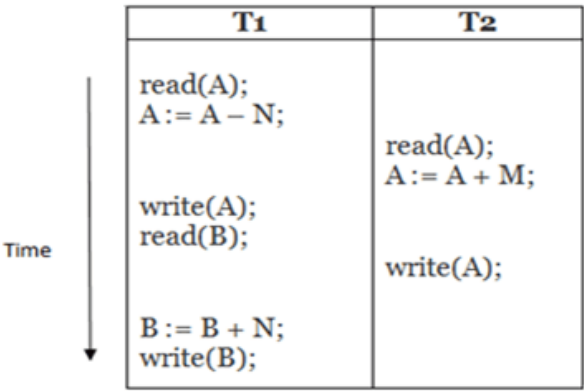
2. Non-Serial Schedules-

- In non-serial schedules,
- Multiple transactions execute concurrently.
- Operations of all the transactions are interleaved or mixed with each other.

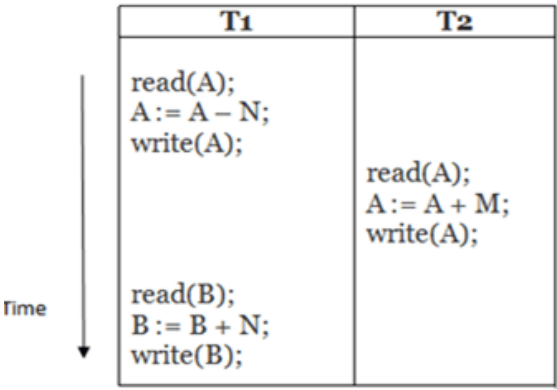
In this schedule,

- There are two transactions T1 and T2 executing concurrently.
- The operations of T1 and T2 are interleaved.
- So, this schedule is an example of a **Non-Serial Schedule**.

Transaction T1	Transaction T2
R (A)	
W (B)	
	R (A)
R (B)	
W (B)	
Commit	
	R (B)
	Commit



Schedule C



Schedule D

Non-Serial Schedules

Types of Non-Serial Schedules

1. Serializable schedule:

- A serializable schedule always leaves the database in consistent state.
- A serial schedule is always a serializable schedule because in serial schedule, a transaction only starts when the other transaction finished execution. However a non-serial schedule needs to be checked for Serializability.
- A non-serial schedule of n number of transactions is said to be serializable schedule, if it is equivalent to the serial schedule of those n transactions.
- Some non-serial schedules may lead to inconsistency of the database.
- Serializability is a concept that helps to identify which non-serial schedules are correct and will maintain the consistency of the database.

Types of Serializability

There are two types of Serializability.

- 1) Conflict Serializability
- 2) View Serializability

- **Conflict Serializable:**

- A schedule is called conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting operations.

Two operations are said to be in conflict, if they satisfy all the following three conditions:

- Both the operations should belong to different transactions.
- Both the operations are working on same data item.
- At least one of the operation is a write operation.

Checking Whether a Schedule is Conflict Serializable Or Not-

Follow the following steps to check whether a given non-serial schedule is conflict serializable or not-

Step-01: Find and list all the conflicting operations.

Step-02: Start creating a precedence graph by drawing one node for each transaction.

Step-03: Draw an edge for each conflict pair such that if $X_i(V)$ and $Y_j(V)$ forms a conflict pair then draw an edge from T_i to T_j .

This ensures that T_i gets executed before T_j .

Step-04: Check if there is any cycle formed in the graph. If there is no cycle found, then the schedule is conflict serializable otherwise not.

EXAMPLE

T_1	T_2	T_3
$R(x)$		$R(y)$ $R(x)$
	$R(y)$ $R(z)$ $w(z)$	$w(y)$
$R(z)$ $w(x)$ $w(z)$		

1) Check conflict pairs in other transactions and draw edge

2) We need to make Precedence graph

(T_1)

(T_2)

i) make nodes

ii) draw edges

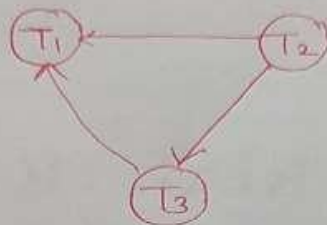
(go to step 1)

(T_3)

3) $R-w$ } conflict pairs, we need to
 $w-R$ } check this pairs of same
 $w-w$ } variables $Rx \rightarrow wx, Ry \rightarrow wy$

T_1	T_2	T_3
$R(z)$		$R(y)$ (ii) $R(z)$ (iii)
	$R(y)$ (iv) $R(z)$ (v)	
$R(z)$ $W(z)$ $W(z)$	$W(z)$ $R(z)$ $W(z)$	$W(y)$ (vi) $R(y)$ (vii) $W(y)$ (viii) (no conflict)

no comparison



conflict ✓
serializable ✓
consistent ✓

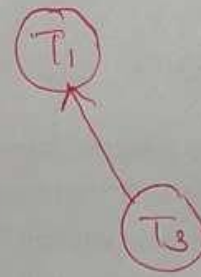
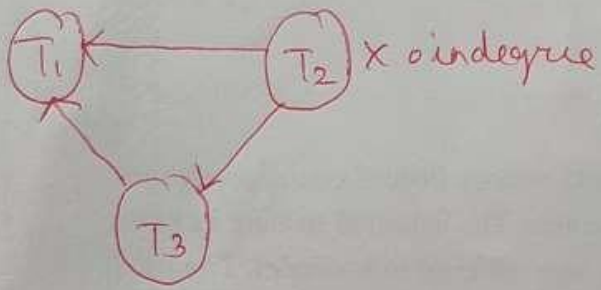
4) check loop/cycle exist in graph or not

In the above graph, there is no loop/no cycle is present. & if no loop & no cycle is present then the graph is conflict serializable.
→ So how to check conflict serializable (we have 3 trans, so 6 possibilities are there)

$T_1 \rightarrow T_2 \rightarrow T_3$
 $T_1 \rightarrow T_3 \rightarrow T_2$
 $T_2 \rightarrow T_1 \rightarrow T_3$
 $T_2 \rightarrow T_3 \rightarrow T_1$
 $T_3 \rightarrow T_1 \rightarrow T_2$
 $T_3 \rightarrow T_2 \rightarrow T_1$

} Six possibilities

To check this possibilities, u find in-degree from the graph and in degree should be zero in the vertex



$$T_2 \rightarrow T_3 \rightarrow T_1$$

so that means the graph is conflict serializable, consistent and serial having order $T_2 \rightarrow T_3 \rightarrow T_1$

Types of Serializability.....

View Serializable schedule

- If a given schedule is found to be view equivalent to some serial schedule, then it is called as a view serializable schedule
- Consider two schedules S1 and S2 each consisting of two transactions T1 and T2.
- Schedules S1 and S2 are called view equivalent if the following three conditions hold true for them-
- “Initial readers must be same for all the data items”.
- “Write-read sequence must be same.”.
- “Final writers must be same for all the data items”.

S ₁		S ₂	
T ₁	T ₂	T ₁	T ₂
R(A)		R(A)	
A = A + 10		A = A + 10	
W(A)		W(A)	
R(B)			R(A)
B = B + 20			A = A + 10
W(B)			W(A)
	R(A)	R(B)	
	A = A + 10	B = B + 20	
	W(A)	W(B)	
	R(B)		W(B)
	B = B + 11		B = B × 11
	W(B)		

- Rules
- 1) Read initial value of data item
 - 2) write final value of data item
 - 3) w - R conflict

	S ₁	S ₂
A	T ₁ T ₂	T ₁ T ₂
B	T ₁ T ₂	T ₁ T ₂

So it is write conflict

Non serializable Schedules

- A non-serial schedule which is not serializable is called as a non-serializable schedule.
- A non-serializable schedule is not guaranteed to produce the same effect as produced by some serial schedule on any consistent database.

Characteristics-

Non-serializable schedules-

- may or may not be consistent
- may or may not be recoverable
- .

Types of Non serializable

1. Recoverable schedule:

- Schedules in which transactions commit only after all transactions whose changes they read commit are called recoverable schedules.
- In other words, if some transaction T_j is reading value updated or written by some other transaction T_i , then the commit of T_j must occur after the commit of T_i .

Here,

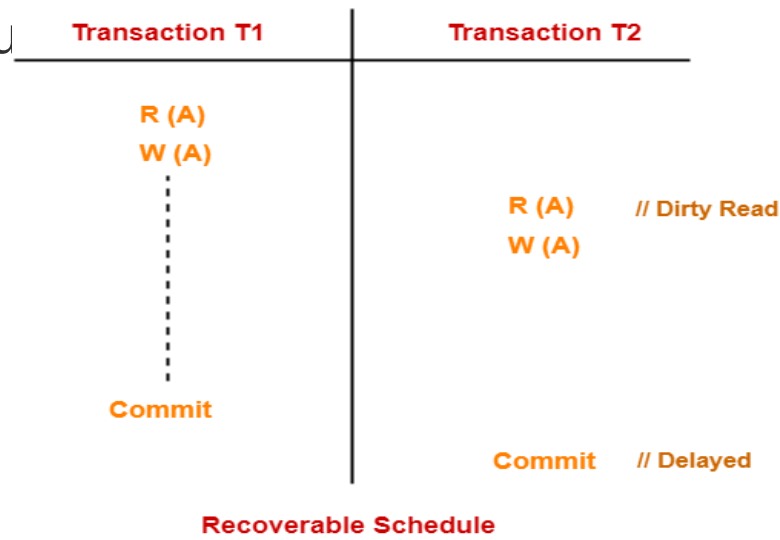
- The commit operation of the transaction that performs the dirty read is delayed.
- This ensures that it still has a chance to recover if the uncommitted transaction fails later.

Types of Non serializable

1. Recoverable schedule.....

Here,

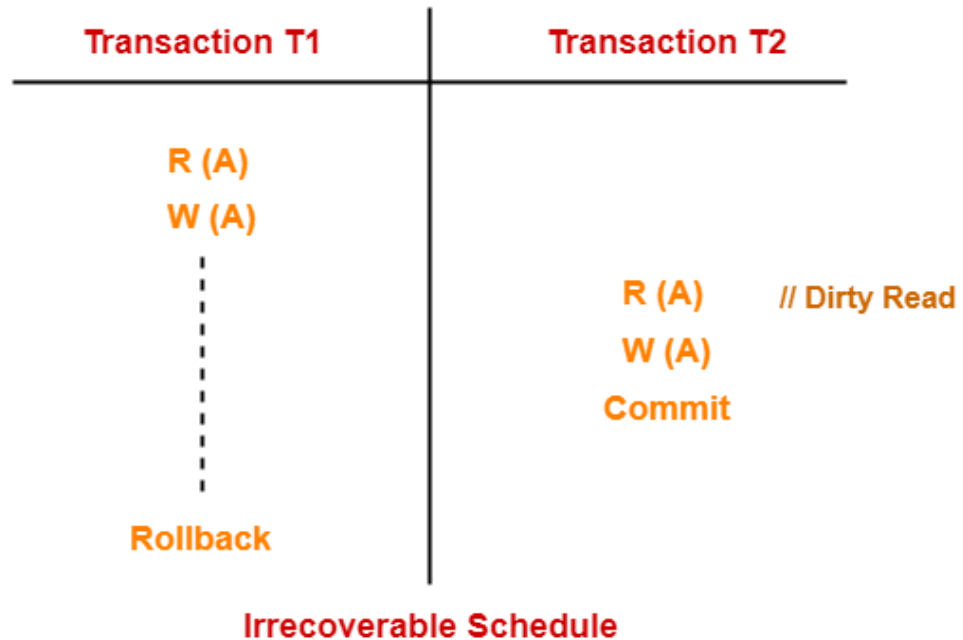
- T2 performs a dirty read operation.
- The commit operation of T2 is delayed till T1 commits or roll backs.
- T1 commits later.
- T2 is now allowed to commit.
- In case, T1 would have rolled back, T2 would have recovered by rolling back.



Types of Non serializable.....

2. Non-recoverable schedule:

- If in a schedule, A transaction performs a dirty read operation from an uncommitted transaction
- And commits before the transaction from which it has read the value then such a schedule is known as an **Irrecoverable Schedule**.

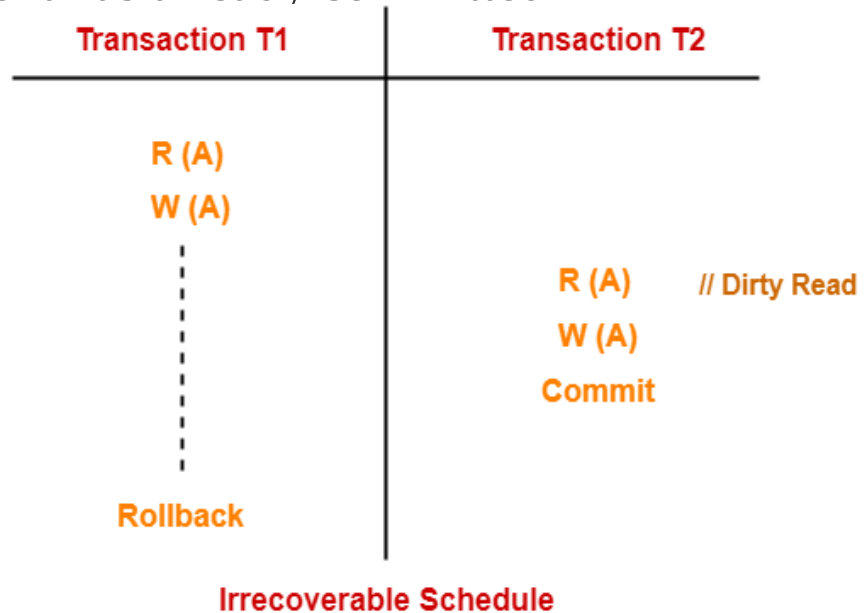


Types of Non serializable.....

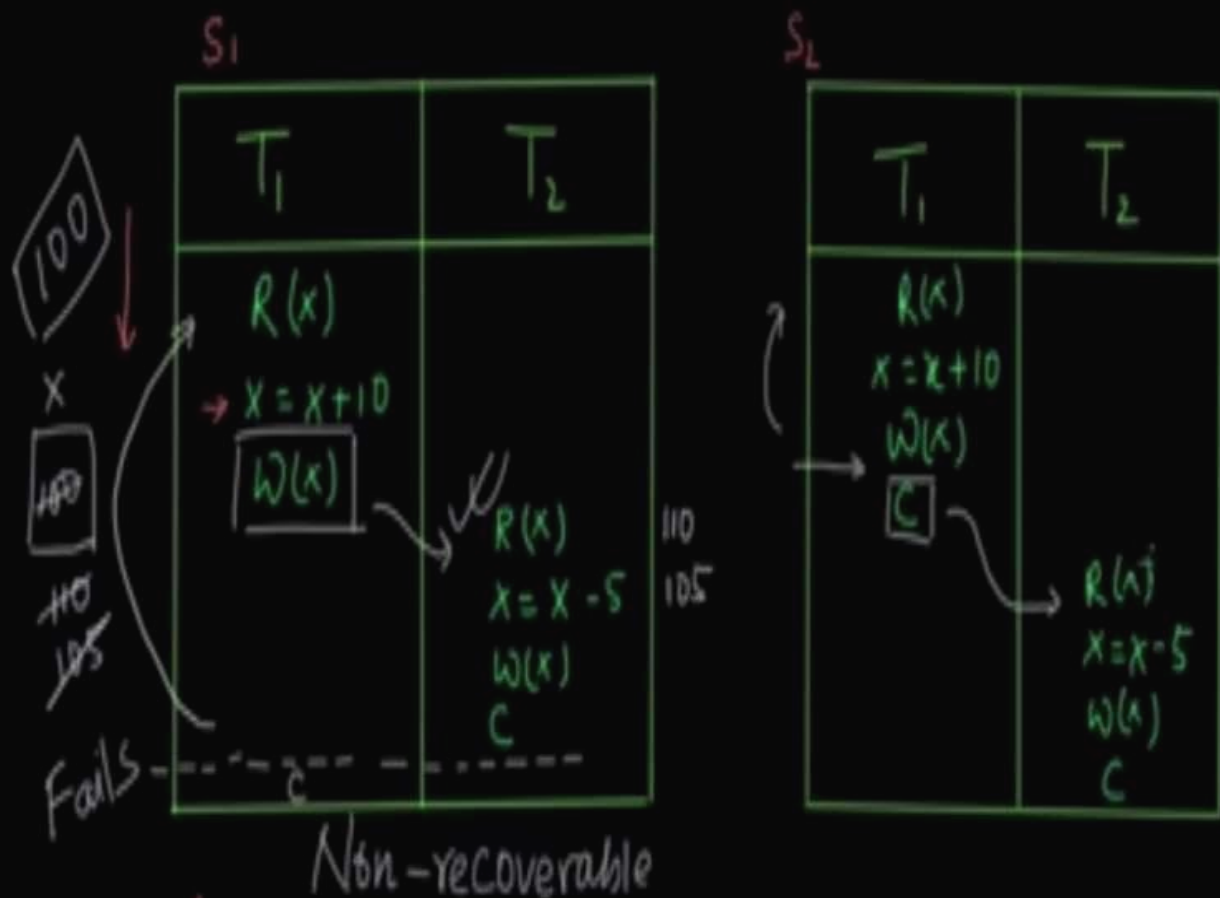
2. Non-recoverable schedule.....

Here,

- T2 performs a dirty read operation.
- T2 commits before T1.
- T1 fails later and roll backs.
- The value that T2 read now stands to be incorrect.
- T2 can not recover since it has already committed.



Recoverable and Nonrecoverable Schedule



* A committed transaction should never be rolled back.

Types of Recoverable schedule

1. Cascading Schedule :

- If in a schedule, failure of one transaction causes several other dependent transactions to rollback or abort, then such a schedule is called as a **Cascading Schedule** or **Cascading Rollback** or **Cascading Abort**.
- It simply leads to the wastage of CPU time.

T1	T2	T3	T4
R (A) W (A)	R (A) W (A)	R (A) W (A)	R (A) W (A)
Failure			

Here,

- Transaction T2 depends on transaction T1.
- Transaction T3 depends on transaction T2.
- Transaction T4 depends on transaction T3.

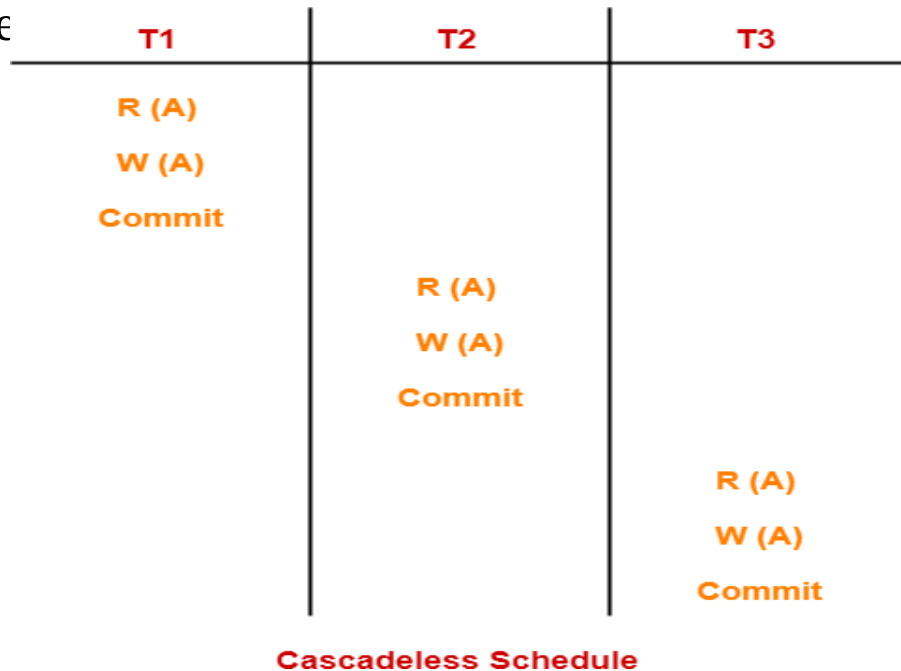
In this schedule,

- The failure of transaction T1 causes the transaction T2 to rollback.
- The rollback of transaction T2 causes the transaction T3 to rollback.
- The rollback of transaction T3 causes the transaction T4 to rollback.
- Such a rollback is called as a **Cascading Rollback**.

Types of Recoverable schedule.....

2. Cascadeless Schedule-

- If in a schedule, a transaction is not allowed to read a data item until the last transaction that has written it is committed or aborted, then such a schedule is called as a **Cascadeless Schedule**.
- In other words, Cascadeless schedule allows only committed read operations. Therefore, it avoids cascading roll back and thus saves CPU time



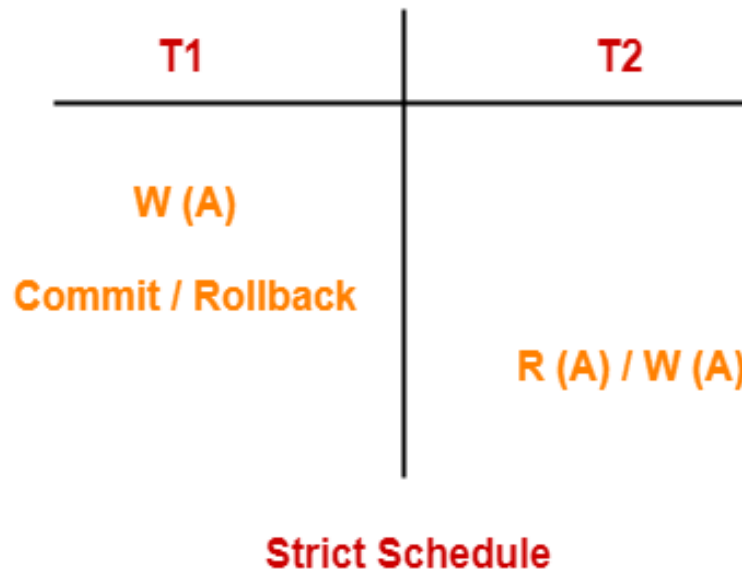
Types of Recoverable schedule.....

3. Strict Schedule-

If in a schedule, a transaction is neither allowed to read nor write a data item until the last transaction that has written it is committed or aborted, then such a schedule is called as a **Strict Schedule**.

In other words,

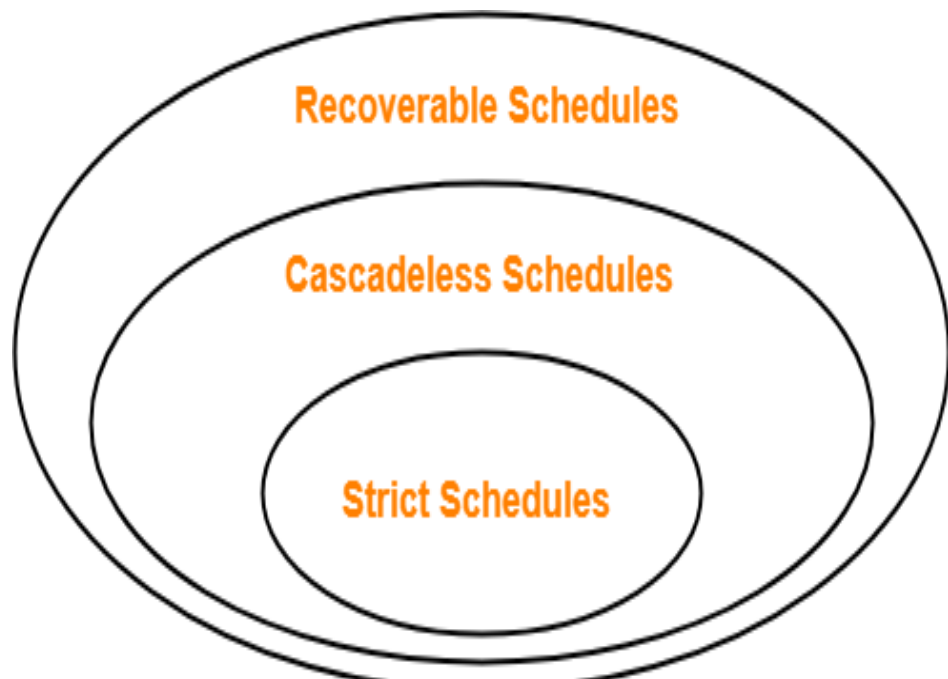
- Strict schedule allows only committed read and write operations.
- Clearly, strict schedule implements more restrictions than cascadeless schedule.



Types of Recoverable schedule.....

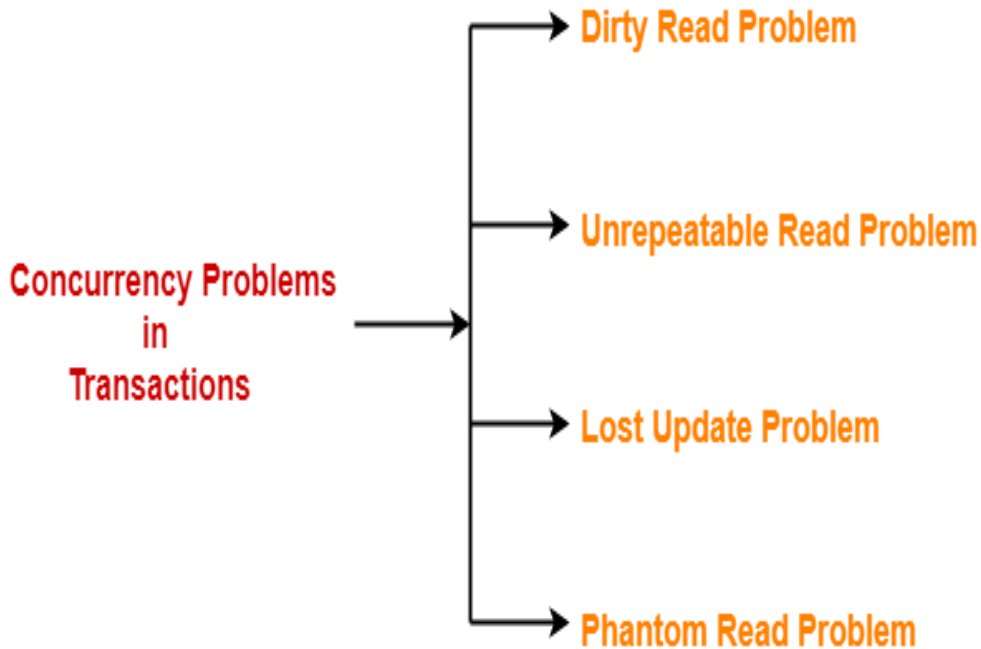
Remember-

- Strict schedules are stricter than Cascadeless schedules.
- All strict schedules are Cascadeless schedules.
- All cascadeless schedules are not strict schedules.



Concurrency Problems in DBMS-

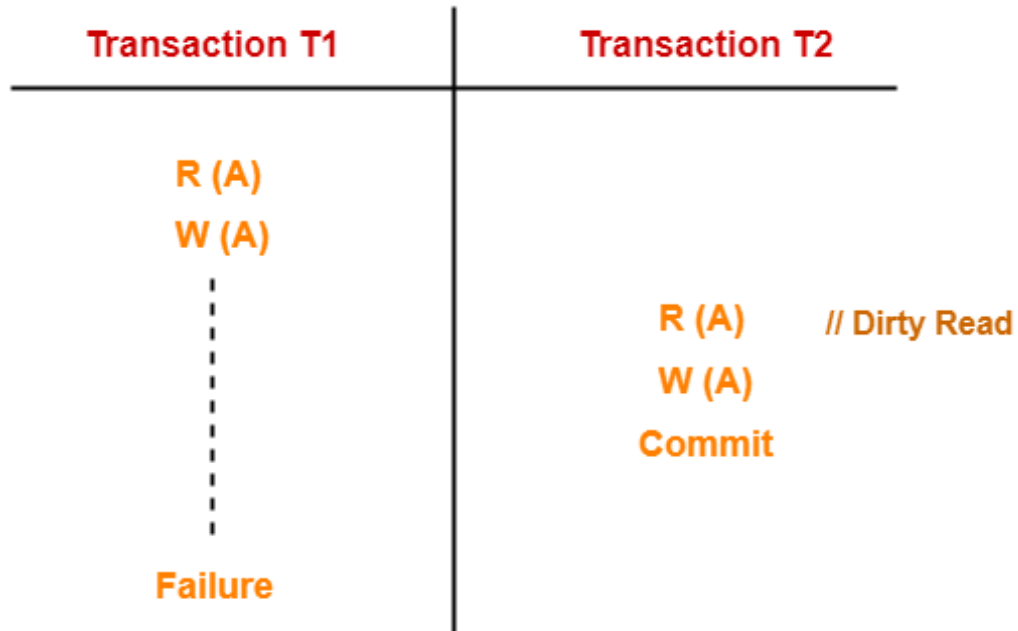
- When multiple transactions execute concurrently in an uncontrolled or unrestricted manner, then it might lead to several problems.
- Such problems are called as **concurrency problems**.



1. Dirty Read Problem-

This read is called as dirty read because-

- There is always a chance that the uncommitted transaction might roll back later.
- Thus, uncommitted transaction might make other transactions read a value that does not even exist.
- This leads to inconsistency of the database.



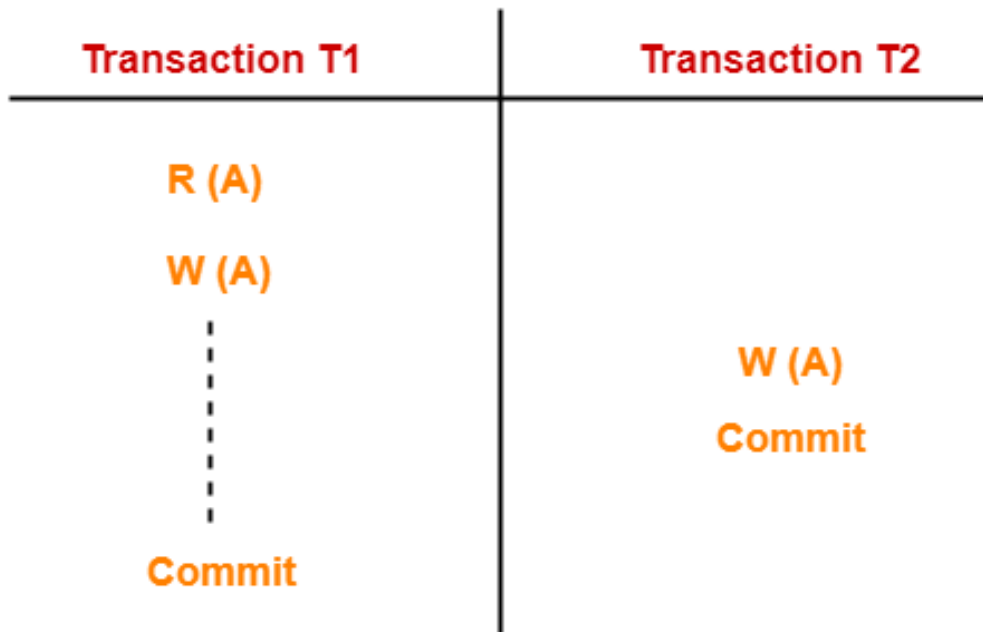
2. Unrepeatable Read Problem-

This problem occurs when a transaction gets to read unrepeatable i.e. different values of the same variable in its different read operations even when it has not updated its value.

Transaction T1	Transaction T2
R (X)	
	R (X)
W (X)	
	R (X) // Unrepeatable Read

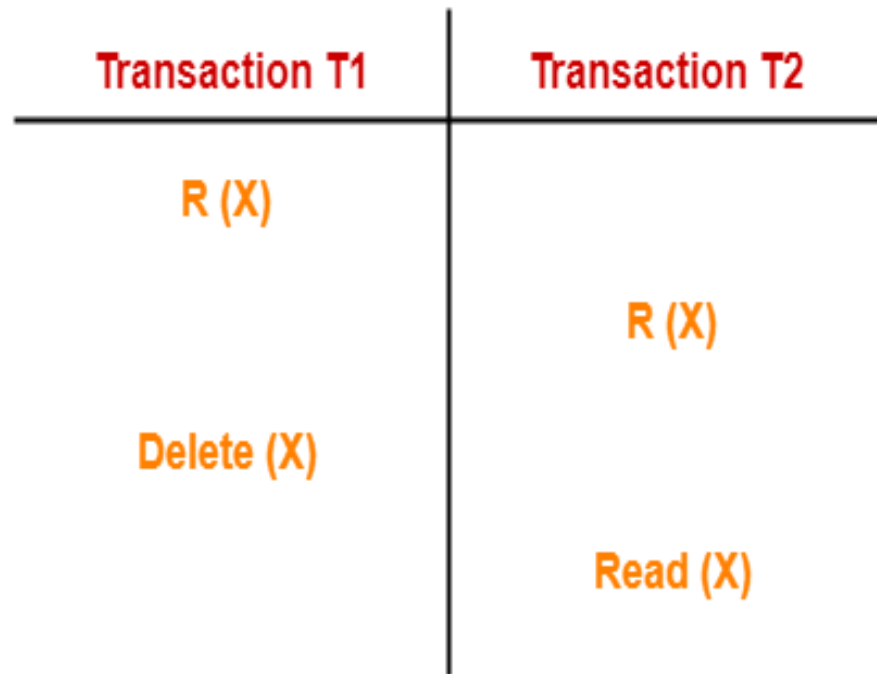
3. Lost Update Problem-

This problem occurs when multiple transactions execute concurrently and updates from one or more transactions get lost.



4. Phantom Read Problem-

This problem occurs when a transaction reads some variable from the buffer and when it reads the same variable later, it finds that the variable does not exist.



Avoiding Concurrency Problems-

- To ensure consistency of the database, it is very important to prevent the occurrence of above problems.
- Concurrency Control Protocols** help to prevent the occurrence of above problems and maintain the consistency of the database.
- Such problems are called as **concurrency problems**.

Concurrency Control

- Concurrency Control in Database Management System is a procedure of managing simultaneous operations without conflicting with each other.
- It ensures that Database transactions are performed concurrently and accurately to produce correct results without violating data integrity of the respective Database.
- When more than one transactions are running simultaneously there are chances of a conflict to occur which can leave database to an inconsistent state.
- To handle these conflicts we need concurrency control in DBMS, which allows transactions to run simultaneously but handles them in such a way so that the integrity of data remains intact.

Concurrency Control

- The technique is used to protect data when multiple users are accessing same data concurrently (same time) is called concurrency control

Concurrency control protocols can be broadly divided into two categories

1) Lock based protocols

A lock is a mechanism to control concurrent access to a data item

Data items can be locked in two modes :

1. *exclusive (X) mode*. Data item can be both read as well as written. X-lock is requested using **lock-X** instruction.

2. *shared (S) mode*. Data item can only be read. S-lock is requested using **lock-S** instruction.

Lock requests are made to the concurrency-control manager by the programmer. Transaction can proceed only after request is granted.

Lock-Based Protocols (Cont.)

Lock-compatibility matrix

	S	X
S	true	false
X	false	false

A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transactions

Any number of transactions can hold shared locks on an item,

But if any transaction holds an exclusive on the item no other transaction may hold any lock on the item.

If a lock cannot be granted, the requesting transaction is made to wait till all incompatible locks held by other transactions have been released. The lock is then granted.

Lock-Based Protocols (Cont.)

Example of a transaction performing locking:

```
 $T_2$ : lock-S(A);  
      read (A);  
      unlock(A);  
      lock-S(B);  
      read (B);  
      unlock(B);  
      display(A+B)
```

Locking as above is not sufficient to guarantee serializability — if A and B get updated in-between the read of A and B , the displayed sum would be wrong.

A **locking protocol** is a set of rules followed by all transactions while requesting and releasing locks. Locking protocols restrict the set of possible schedules.

Concurrency control protocols can be broadly divided into two categories

1) Lock based protocols.....

- Lock-compatibility Matrix :
- S X S TRUE FALSE X FALSE FALSE
- - A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transaction
 - Any number of transactions can hold shared locks on an item, But if any transaction holds an exclusive on the item no other transaction may hold any lock on the item

Lock Based Protocol

- If a lock cannot be granted, the requesting transaction is made to wait till all incompatible locks held by other transaction have been released. The lock is then granted.

- Example:

```
T1: lock-S(A);
```

```
// Grant-S(A,
```

```
T1) read (A);
```

```
unlock(A);
```

```
lock-S(B);
```

```
// Grant-S(B,T1) read (B);
```

```
unlock(B);
```

```
display(A+B
```

Pitfalls of Lock Based Protocol

- Locking as above is not sufficient to guarantee serializability - if A and B get updated in-between the read of A and B, the displayed sum would be wrong.
- A locking protocol is a set of rules followed by all transactions while requesting and releasing locks. Locking protocols restrict the set of possible schedules.
- Locking as above is not sufficient to guarantee serializability - if A and B get updated in-between the read of A and B, the displayed sum would be wrong.
- A locking protocol is a set of rules followed by all transactions while requesting and releasing locks. Locking protocols restrict the set of possible schedules.

Pitfalls of Lock Based Protocol

- Starvation is also possible if concurrency control manager is badly designed.
- For example: – A transaction may be waiting for an X-lock on an item, while a sequence of other transactions request and are granted
- an S-lock on the same item. – The same transaction is repeatedly rolled back due to deadlocks.
- • Concurrency control manager can be designed to prevent starvation.

Two Phase Locking Protocol

- This is a protocol which ensures conflict-serializable schedules.
- Phase 1: Growing Phase – transaction may obtain locks – transaction may not release locks
- Phase 2: Shrinking Phase – transaction may release locks – transaction may not obtain locks
- The protocol assures serializability. It can be proved that the transactions can be serialized in the order of their lock points (i.e. the point where a transaction acquired its final lock).

Two Phase Locking Protocol

Two-phase locking does not ensure freedom from deadlocks • Cascading roll-back is possible under two-phase locking. To avoid this, follow a modified protocol called strict two-phase locking. Here a transaction must hold all its exclusive locks till it commits/aborts. • Rigorous two-phase locking is even stricter: here all locks are held till commit/abort. In this protocol transactions can be serialized in the order in which they commit.

Two Phase Locking Protocol

Two-phase locking does not ensure freedom from deadlocks • Cascading roll-back is possible under two-phase locking. To avoid this, follow a modified protocol called strict two-phase locking. Here a transaction must hold all its exclusive locks till it commits/aborts. • Rigorous two-phase locking is even stricter: here all locks are held till commit/abort. In this protocol transactions can be serialized in the order in which they commit.

The Two-Phase Locking Protocol

This protocol ensures conflict-serializable schedules.

Phase 1: Growing Phase

- Transaction may obtain locks

- Transaction may not release locks

Phase 2: Shrinking Phase

- Transaction may release locks

- Transaction may not obtain locks

The protocol assures serializability. It can be proved that the transactions can be serialized in the order of their **lock points** (i.e., the point where a transaction acquired its final lock).

The Two-Phase Locking Protocol (Cont.)

There can be conflict serializable schedules that cannot be obtained if two-phase locking is used.

However, in the absence of extra information (e.g., ordering of access to data), two-phase locking is needed for conflict serializability in the following sense:

Given a transaction T_i that does not follow two-phase locking, we can find a transaction T_j that uses two-phase locking, and a schedule for T_i and T_j that is not conflict serializable.

Lock Conversions

Two-phase locking with lock conversions:

- **First Phase:**

- can acquire a lock-S on item
- can acquire a lock-X on item
- can convert a lock-S to a lock-X (upgrade)

- **Second Phase:**

- can release a lock-S
- can release a lock-X
- can convert a lock-X to a lock-S (downgrade)

This protocol assures serializability. But still relies on the programmer to insert the various locking instructions.

Automatic Acquisition of Locks

A transaction T_i issues the standard read/write instruction, without explicit locking calls.

The operation **read**(D) is processed as:

```
if  $T_i$  has a lock on  $D$ 
  then
    read( $D$ )
  else begin
    if necessary wait until no other
      transaction has a lock-X on  $D$ 
    grant  $T_i$  a lock-S on  $D$ ;
    read( $D$ )
  end
```

Automatic Acquisition of Locks (Cont.)

write(D) is processed as:

```
    if  $T_i$  has a lock-X on  $D$ 
    then
        write( $D$ )
    else begin
        if necessary wait until no other transaction has any lock
on  $D$ ,
        if  $T_i$  has a lock-S on  $D$ 
            then
                upgrade lock on  $D$  to lock-X
            else
                grant  $T_i$  a lock-X on  $D$ 
        write( $D$ )
    end;
```

All locks are released after commit or abort

Deadlocks

Deadlock • A set of processes is deadlocked if each process in the set is waiting for an event that only another process in the set can cause. T_i T_j

Consider the partial schedule

T_3	T_4
lock-x (B)	
read (B)	
$B := B - 50$	
write (B)	
	lock-s (A)
	read (A)
	lock-s (B)
lock-x (A)	

Neither T_3 nor T_4 can make progress — executing **lock-S(B)** causes T_4 to wait for T_3 to release its lock on B , while executing **lock-X(A)** causes T_3 to wait for T_4 to release its lock on A .

Such a situation is called a **deadlock**.

To handle a deadlock one of T_3 or T_4 must be rolled back and its locks released.

Deadlocks (Cont.)

Two-phase locking *does not* ensure freedom from deadlocks.

In addition to deadlocks, there is a possibility of **starvation**.

Starvation occurs if the concurrency control manager is badly designed. For example:

- A transaction may be waiting for an X-lock on an item, while a sequence of other transactions request and are granted an S-lock on the same item.

- The same transaction is repeatedly rolled back due to deadlocks.

Concurrency control manager can be designed to prevent starvation.

Deadlocks (Cont.)

The potential for deadlock exists in most locking protocols. Deadlocks are a necessary evil.

When a deadlock occurs there is a possibility of cascading roll-backs.

Cascading roll-back is possible under two-phase locking. To avoid this, follow a modified protocol called **strict two-phase locking** -- a transaction must hold all its exclusive locks till it commits/aborts.

Rigorous two-phase locking is even stricter. Here, *all* locks are held till commit/abort. In this protocol transactions can be serialized in the order in which they commit.

Implementation of Locking

A **lock manager** can be implemented as a separate process to which transactions send lock and unlock requests

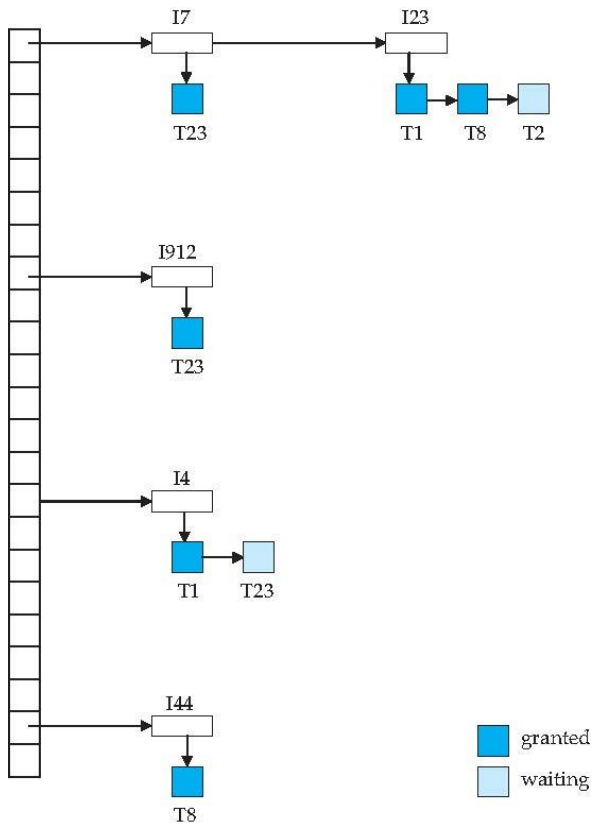
The lock manager replies to a lock request by sending a lock grant messages (or a message asking the transaction to roll back, in case of a deadlock)

The requesting transaction waits until its request is answered

The lock manager maintains a data-structure called a **lock table** to record granted locks and pending requests

The lock table is usually implemented as an in-memory hash table indexed on the name of the data item being locked

Lock Table



Dark blue rectangles indicate granted locks; light blue indicate waiting requests

Lock table also records the type of lock granted or requested

New request is added to the end of the queue of requests for the data item, and granted if it is compatible with all earlier locks

Unlock requests result in the request being deleted, and later requests are checked to see if they can now be granted

If transaction aborts, all waiting or granted requests of the transaction are deleted

lock manager may keep a list of locks held by each transaction, to implement this efficiently

Deadlock Handling

System is deadlocked if there is a set of transactions such that every transaction in the set is waiting for another transaction in the set.

Deadlock prevention protocols ensure that the system will *never* enter into a deadlock state. Some prevention strategies :

- Require that each transaction locks all its data items before it begins execution (predeclaration).

- Impose partial ordering of all data items and require that a transaction can lock data items only in the order specified by the partial order.

More Deadlock Prevention Strategies

Following schemes use transaction timestamps for the sake of deadlock prevention alone.

wait-die scheme — non-preemptive

older transaction may wait for younger one to release data item. (older means smaller timestamp) Younger transactions never wait for older ones; they are rolled back instead.

a transaction may die several times before acquiring needed data item

wound-wait scheme — preemptive

older transaction *wounds* (forces rollback) of younger transaction instead of waiting for it. Younger transactions may wait for older ones.

may be fewer rollbacks than *wait-die* scheme.

Deadlock prevention (Cont.)

Both in *wait-die* and in *wound-wait* schemes, a rolled back transactions is restarted with its original timestamp. Older transactions thus have precedence over newer ones, and starvation is hence avoided.

Timeout-Based Schemes:

a transaction waits for a lock only for a specified amount of time. If the lock has not been granted within that time, the transaction is rolled back and restarted,

Thus, deadlocks are not possible

simple to implement; but starvation is possible. Also difficult to determine good value of the timeout interval.

Deadlock Detection

Deadlocks can be described as a *wait-for graph*, which consists of a pair $G = (V, E)$,

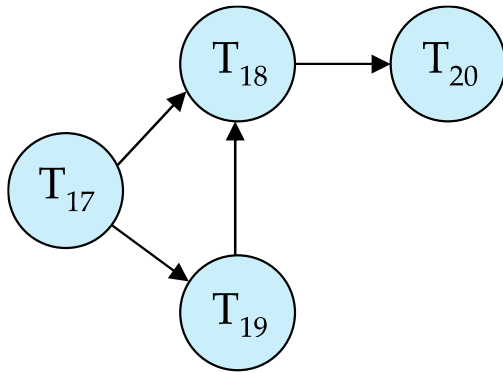
V is a set of vertices (all the transactions in the system)

E is a set of edges; each element is an ordered pair $T_i \rightarrow T_j$.

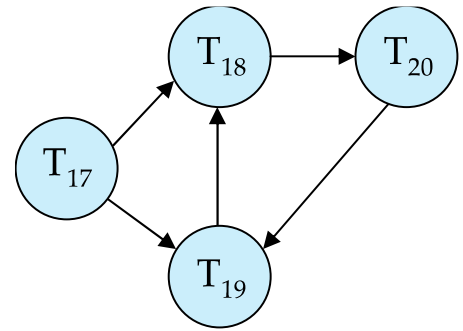
If $T_i \rightarrow T_j$ is in E , then there is a directed edge from T_i to T_j , implying that T_i is waiting for T_j to release a data item.

When T_i requests a data item currently being held by T_j , then the edge $T_i \rightarrow T_j$ is inserted in the wait-for graph. This edge is removed only when T_j is no longer holding a data item needed by T_i .

Deadlock Detection (Cont.)



Wait-for graph without a cycle



Wait-for graph with a cycle

Deadlock Recovery

When deadlock is detected :

Some transaction will have to rolled back (made a victim) to break deadlock. Select that transaction as victim that will incur minimum cost.

Rollback -- determine how far to roll back transaction

Total rollback: Abort the transaction and then restart it.

More effective to roll back transaction only as far as necessary to break deadlock.

Starvation happens if same transaction is always chosen as victim. Include the number of rollbacks in the cost factor to avoid starvation

Multiple Granularity

Allow data items to be of various sizes and define a hierarchy of data granularities, where the small granularities are nested within larger ones

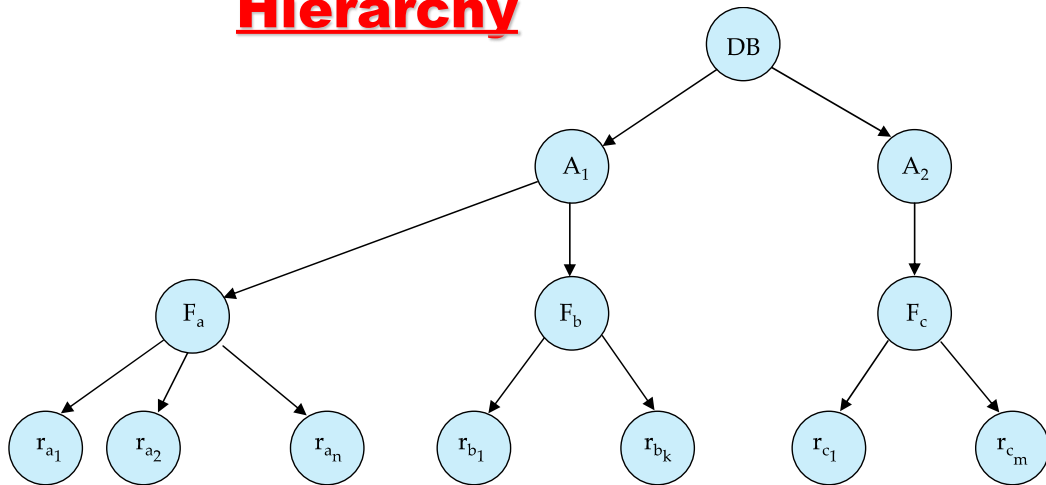
Can be represented graphically as a tree.

When a transaction locks a node in the tree *explicitly*, it *implicitly* locks all the node's descendents in the same mode.

Granularity of locking (level in tree where locking is done):

fine granularity (lower in tree): high concurrency, high locking overhead

Example of Granularity Hierarchy



The levels, starting from the coarsest (top) level are
database
area
file
record

Intention Lock Modes

In addition to S and X lock modes, there are three additional lock modes with multiple granularity:

intention-shared (IS): indicates explicit locking at a lower level of the tree but only with shared locks.

intention-exclusive (IX): indicates explicit locking at a lower level with exclusive or shared locks

shared and intention-exclusive (SIX): the subtree rooted by that node is locked explicitly in shared mode and explicit locking is being done at a lower level with exclusive-mode locks.

intention locks allow a higher level node to be locked in S or X mode without having to check all descendent nodes.

Compatibility Matrix with Intention Lock Modes

The compatibility matrix for all lock modes is:

	IS	IX	S	SIX	X
IS	true	true	true	true	false
IX	true	true	false	false	false
S	true	false	true	false	false
SIX	true	false	false	false	false
X	false	false	false	false	false

Multiple Granularity Locking Scheme

Transaction T_i can lock a node Q , using the following rules:

1. The lock compatibility matrix must be observed.
2. The root of the tree must be locked first, and may be locked in any mode.
3. A node Q can be locked by T_i in S or IS mode only if the parent of Q is currently locked by T_i in either IX or IS mode.
4. A node Q can be locked by T_i in X, SIX, or IX mode only if the parent of Q is currently locked by T_i in either IX or SIX mode.
5. T_i can lock a node only if it has not previously unlocked any node (that is, T_i is two-phase).
6. T_i can unlock a node Q only if none of the children of Q are currently locked by T_i .

Observe that locks are acquired in root-to-leaf order, whereas they are released in leaf-to-root order.

Lock granularity escalation: in case there are too many locks at a particular level, switch to higher granularity S or X lock

Timestamp-Based Protocols

Each transaction is issued a timestamp when it enters the system. If an old transaction T_i has time-stamp $TS(T_i)$, a new transaction T_j is assigned time-stamp $TS(T_j)$ such that $TS(T_i) < TS(T_j)$.

The protocol manages concurrent execution such that the time-stamps determine the serializability order.

In order to assure such behavior, the protocol maintains for each data Q two timestamp values:

W-timestamp(Q) is the largest time-stamp of any transaction that executed **write**(Q) successfully.

R-timestamp(Q) is the largest time-stamp of any transaction that executed **read**(Q) successfully.

Timestamp-Based Protocols (Cont.)

The timestamp ordering protocol ensures that any conflicting **read** and **write** operations are executed in timestamp order.

Suppose a transaction T_i issues a **read**(Q)

1. If $TS(T_i) \leq \mathbf{W}\text{-timestamp}(Q)$, then T_i needs to read a value of Q that was already overwritten.
■ Hence, the **read** operation is rejected, and T_i is rolled back.
2. If $TS(T_i) \geq \mathbf{W}\text{-timestamp}(Q)$, then the **read** operation is executed, and $\mathbf{R}\text{-timestamp}(Q)$ is set to $\mathbf{max}(\mathbf{R}\text{-timestamp}(Q), TS(T_i))$.

Timestamp-Based Protocols (Cont.)

Suppose that transaction T_i issues **write**(Q).

1. If $TS(T_i) < R\text{-timestamp}(Q)$, then the value of Q that T_i is producing was needed previously, and the system assumed that that value would never be produced.
■ Hence, the **write** operation is rejected, and T_i is rolled back.
2. If $TS(T_i) < W\text{-timestamp}(Q)$, then T_i is attempting to write an obsolete value of Q .
■ Hence, this **write** operation is rejected, and T_i is rolled back.
3. Otherwise, the **write** operation is executed, and $W\text{-timestamp}(Q)$ is set to $TS(T_i)$.

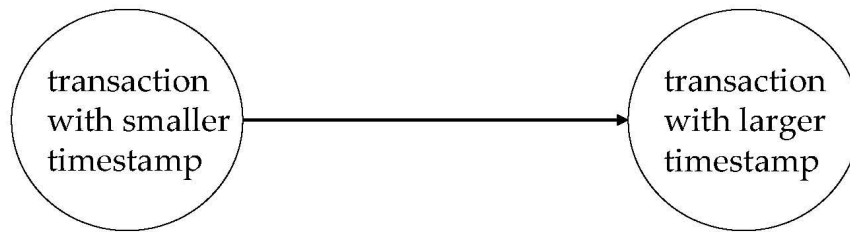
Example Use of the Protocol

A partial schedule for several data items for transactions with timestamps 1, 2, 3, 4, 5

T_1	T_2	T_3	T_4	T_5
				read (X)
read (Y)	read (Y)	write (Y) write (Z)		
	read (Z) abort			read (Z)
read (X)		write (W) abort	read (W)	
				write (Y) write (Z)

Correctness of Timestamp-Ordering Protocol

The timestamp-ordering protocol guarantees serializability since all the arcs in the precedence graph are of the form:



Thus, there will be no cycles in the precedence graph. Timestamp protocol ensures freedom from deadlock as no transaction ever waits.

But the schedule may not be cascade-free, and may not even be recoverable.

Recoverability and Cascade Freedom

Problem with timestamp-ordering protocol:

Suppose T_i aborts, but T_j has read a data item written by T_i

Then T_j must abort; if T_j had been allowed to commit earlier, the schedule is not recoverable.

Further, any transaction that has read a data item written by T_j must abort

This can lead to cascading rollback --- that is, a chain of rollbacks

Solution 1:

A transaction is structured such that its writes are all performed at the end of its processing

All writes of a transaction form an atomic action; no transaction may execute while a transaction is being written

A transaction that aborts is restarted with a new timestamp

Solution 2: Limited form of locking: wait for data to be committed before reading it

Solution 3: Use commit dependencies to ensure recoverability

Thomas' Write Rule

Modified version of the timestamp-ordering protocol in which obsolete **write** operations may be ignored under certain circumstances.

When T_i attempts to write data item Q , if $TS(T_i) < W\text{-timestamp}(Q)$, then T_i is attempting to write an obsolete value of $\{Q\}$.

Rather than rolling back T_i as the timestamp ordering protocol would have done, this **{write}** operation can be ignored.

Otherwise this protocol is the same as the timestamp ordering protocol.

Validation-Based Protocol

Execution of transaction T_i is done in three phases.

1. **Read and execution phase:** Transaction T_i writes only to temporary local variables
2. **Validation phase:** Transaction T_i performs a "validation test" to determine if local variables can be written without violating serializability.
3. **Write phase:** If T_i is validated, the updates are applied to the database; otherwise, T_i is rolled back.

The three phases of concurrently executing transactions can be interleaved, but each transaction must go through the three phases in that order.

Assume for simplicity that the validation and write phase occur together, atomically and serially

I.e., only one transaction executes validation/write at a time.

Also called as **optimistic concurrency control** since transaction executes fully in the hope that all will go well during validation

Validation-Based Protocol (Cont.)

Each transaction T_i has 3 timestamps

Start(T_i) : the time when T_i started its execution

Validation(T_i): the time when T_i entered its validation phase

Finish(T_i) : the time when T_i finished its write phase

Serializability order is determined by timestamp given at validation time; this is done to increase concurrency.

Thus, $TS(T_i)$ is given the value of Validation(T_i).

This protocol is useful and gives greater degree of concurrency if probability of conflicts is low.

because the serializability order is not pre-decided, and relatively few transactions will have to be rolled back.

Validation Test for Transaction T_j

If for all T_i with $TS(T_i) < TS(T_j)$ either one of the following condition holds:

$finish(T_i) < start(T_j)$

$start(T_j) < finish(T_i) < validation(T_j)$ and the set of data items written by T_i does not intersect with the set of data items read by T_j .

then validation succeeds and T_j can be committed.

Otherwise, validation fails and T_j is aborted.

Justification: Either the first condition is satisfied, and there is no overlapped execution, or the second condition is satisfied and

- the writes of T_j do not affect reads of T_i since they occur after T_i has finished its reads.
- the writes of T_i do not affect reads of T_j since T_j does not read any item written by T_i .

Schedule Produced by Validation

Example of schedule produced using validation

T_{25}	T_{26}
read (B)	read (B) $B := B - 50$ read (A) $A := A + 50$
read (A) $\langle \text{validate} \rangle$ display ($A + B$)	$\langle \text{validate} \rangle$ write (B) write (A)

Multiversion Schemes

Multiversion schemes keep old versions of data item to increase concurrency.

- Multiversion Timestamp Ordering

- Multiversion Two-Phase Locking

Each successful **write** results in the creation of a new version of the data item written.

Use timestamps to label versions.

When a **read**(Q) operation is issued, select an appropriate version of Q based on the timestamp of the transaction, and return the value of the selected version.

▪ 1 , . ,

Multiversion Timestamp Ordering

Each data item Q has a sequence of versions $\langle Q_1, Q_2, \dots, Q_m \rangle$. Each version Q_k contains three data fields:

Content -- the value of version Q_k .

W-timestamp(Q_k) -- timestamp of the transaction that created (wrote) version Q_k

R-timestamp(Q_k) -- largest timestamp of a transaction that successfully read version Q_k

When a transaction T_i creates a new version Q_k of Q , Q_k 's W-timestamp and R-timestamp are initialized to $TS(T_i)$. R-timestamp of Q_k is updated whenever a transaction T_j reads Q_k , and $TS(T_j) > R\text{-timestamp}(Q_k)$.

Multiversion Timestamp Ordering (Cont)

Suppose that transaction T_i issues a **read**(Q) or **write**(Q) operation. Let Q_k denote the version of Q whose write timestamp is the largest write timestamp less than or equal to $TS(T_i)$.

1. If transaction T_i issues a **read**(Q), then the value returned is the content of version Q_k .
2. If transaction T_i issues a **write**(Q)
 1. if $TS(T_i) < R\text{-timestamp}(Q_k)$, then transaction T_i is rolled back.
 2. if $TS(T_i) = W\text{-timestamp}(Q_k)$, the contents of Q_k are overwritten
 3. else a new version of Q is created.

Observe that

Reads always succeed

A write by T_i is rejected if some other transaction T_j that (in the serialization order defined by the timestamp values) should read

T_i 's write, has already read a version created by a transaction older than T_i .

Protocol guarantees serializability

Multiversion Two-Phase Locking

Differentiates between read-only transactions and update transactions

Update transactions acquire read and write locks, and hold all locks up to the end of the transaction. That is, update transactions follow rigorous two-phase locking.

Each successful **write** results in the creation of a new version of the data item written.

Each version of a data item has a single timestamp whose value is obtained from a counter **ts-counter** that is incremented during commit processing.

Read-only transactions are assigned a timestamp by reading the current value of **ts-counter** before they start execution; they follow the multiversion timestamp-ordering protocol for performing reads.

Multiversion Two-Phase Locking (Cont.)

When an update transaction wants to read a data item:
it obtains a shared lock on it, and reads the latest version.

When it wants to write an item

it obtains X lock on; it then creates a new version of the item and sets this version's timestamp to ∞ .

When update transaction T_i completes, commit processing occurs:

T_i sets timestamp on the versions it has created to **ts-counter** + 1

T_i increments **ts-counter** by 1

Read-only transactions that start after T_i increments **ts-counter** will see the values updated by T_i .

Read-only transactions that start before T_i increments the **ts-counter** will see the value before the updates by T_i .

Only serializable schedules are produced.

MVCC: **Implementation** **Issues**

Creation of multiple versions increases storage overhead

- Extra tuples

- Extra space in each tuple for storing version information

Versions can, however, be garbage collected

- E.g. if Q has two versions Q5 and Q9, and the oldest active transaction has timestamp > 9 , then Q5 will never be required again

Snapshot Isolation

Motivation: Decision support queries that read large amounts of data have concurrency conflicts with OLTP transactions that update a few rows

Poor performance results

Solution 1: Give logical “snapshot” of database state to read only transactions, read-write transactions use normal locking

Multiversion 2-phase locking

Works well, but how does system know a transaction is read only?

Solution 2: Give snapshot of database state to every transaction, updates alone use 2-phase locking to guard against concurrent updates

Problem: variety of anomalies such as lost update can result

Partial solution: snapshot isolation level (next slide)

Proposed by Berenson et al, SIGMOD 1995

Variants implemented in many database systems

E.g. Oracle, PostgreSQL, SQL Server 2005

Snapshot Isolation

A transaction T1 executing with Snapshot Isolation
takes snapshot of committed data at start
always reads/modifies data in its own snapshot
updates of concurrent transactions are not
visible to T1

writes of T1 complete when it commits

First-committer-wins rule:

Commits only if no other concurrent
transaction has already written data that T1
intends to write.

T1	T2	T3
W(Y := 1) Commit		
	Start R(X) → 0 R(Y) → 1	
		W(X:=2) W(Z:=3) Commit
	R(Z) → 0 R(Y) → 1 W(X:=3) Commit-Req Abort	

Concurrent updates not visible
Own updates are visible
Not first-committer of X
Serialization error, T2 is rolled back

Snapshot Read

- Concurrent updates invisible to snapshot read

$X_0 = 100, Y_0 = 0$

T_1 deposits 50 in Y	T_2 withdraws 50 from X
$r_1(X_0, 100)$ $r_1(Y_0, 0)$ $w_1(Y_1, 50)$ $r_1(X_0, 100)$ (update by T_2 not seen) $r_1(Y_1, 50)$ (can see its own updates)	$r_2(Y_0, 0)$ $r_2(X_0, 100)$ $w_2(X_2, 50)$ $r_2(Y_0, 0)$ (update by T_1 not seen)

$X_2 = 50, Y_1 = 50$

Snapshot Write: First Committer Wins

$X_0 = 100$

T_1 deposits 50 in X	T_2 withdraws 50 from X
$r_1(X_0, 100)$ $w_1(X_1, 150)$ $commit_1$	$r_2(X_0, 100)$ $w_2(X_2, 50)$ $commit_2$ (Serialization Error T_2 is rolled back)

$X_1 = 150$

Variant: **First-updater-wins**

Check for concurrent updates when write occurs by locking item

But lock should be held till all concurrent transactions have finished

(Oracle uses this plus some extra features)

Differs only in when abort occurs, otherwise equivalent

Benefits of SI

Reading is *never* blocked,
and also doesn't block other txns activities

Performance similar to Read Committed

Avoids the usual anomalies

- No dirty read

- No lost update

- No non-repeatable read

- Predicate based selects are repeatable (no phantoms)

Problems with SI

- SI does not always give serializable executions

 - Serializable: among two concurrent txns, one sees the effects of the other

 - In SI: neither sees the effects of the other

- Result: Integrity constraints can be violated

Snapshot Isolation

E.g. of problem with SI

T1: $x := y$

T2: $y := x$

Initially $x = 3$ and $y = 17$

Serial execution: $x = ??$, $y = ??$

if both transactions start at the same time, with snapshot isolation:

$x = ??$, $y = ??$

Called **skew write**

Skew also occurs with inserts

E.g:

Find max order number among all orders

Create a new order with order number = previous max + 1

Snapshot Isolation

Anomalies

SI breaks serializability when txns modify *different* items, each based on a previous state of the item the other modified

Not very common in practice

E.g., the TPC-C benchmark runs correctly under SI

when txns conflict due to modifying different data, there is usually also a shared item they both modify too (like a total quantity) so SI will abort one of them

But does occur

Application developers should be careful about write skew

SI can also cause a read-only transaction anomaly, where read-only transaction may see an inconsistent state even if updaters are serializable

We omit details

Using snapshots to verify primary/foreign key integrity can lead to inconsistency

Integrity constraint checking usually done outside of snapshot

SI In Oracle and PostgreSQL

Warning: SI used when isolation level is set to serializable, by Oracle, and PostgreSQL versions prior to 9.1

PostgreSQL's implementation of SI (versions prior to 9.1) described in Section 26.4.1.3

Oracle implements "first updater wins" rule (variant of "first committer wins")
concurrent writer check is done at time of write, not at commit time

Allows transactions to be rolled back earlier

Oracle and PostgreSQL < 9.1 do not support true serializable execution

PostgreSQL 9.1 introduced new protocol called "Serializable Snapshot Isolation" (SSI)

Which guarantees true serializability including handling predicate reads (coming up)

SI In Oracle and PostgreSQL

Can sidestep SI for specific queries by using **select .. for update** in Oracle and PostgreSQL

E.g.,

1. **select max(orderno) from orders for update**
2. read value into local variable maxorder
3. insert into orders (maxorder+1, ...)

Select for update (SFU) treats all data read by the query as if it were also updated, preventing concurrent updates

Does not always ensure serializability since phantom phenomena can occur (coming up)

In PostgreSQL versions < 9.1, SFU locks the data item, but releases locks when the transaction completes, even if other concurrent transactions are active

Not quite same as SFU in Oracle, which keeps locks until all concurrent transactions have completed

Insert and Delete Operations

If two-phase locking is used :

A **delete** operation may be performed only if the transaction deleting the tuple has an exclusive lock on the tuple to be deleted.

A transaction that inserts a new tuple into the database is given an X-mode lock on the tuple

Insertions and deletions can lead to the **phantom phenomenon**.

A transaction that scans a relation

(e.g., find sum of balances of all accounts in Perryridge)

and a transaction that inserts a tuple in the relation

(e.g., insert a new account at Perryridge)

(conceptually) conflict in spite of not accessing any tuple in common.

If only tuple locks are used, non-serializable schedules can result

Insert and Delete Operations (Cont.)

The transaction scanning the relation is reading information that indicates what tuples the relation contains, while a transaction inserting a tuple updates the same information.

The conflict should be detected, e.g. by locking the information.

One solution:

Associate a data item with the relation, to represent the information about what tuples the relation contains.

Transactions scanning the relation acquire a shared lock in the data item,

Transactions inserting or deleting a tuple acquire an exclusive lock on the data item. (Note: locks on the data item do not conflict with locks on individual tuples.)

Above protocol provides very low concurrency for insertions/deletions.

Index locking protocols provide higher concurrency while preventing the phantom phenomenon, by requiring locks on certain index buckets.

Index Locking Protocol

Index locking protocol:

- Every relation must have at least one index.

- A transaction can access tuples only after finding them through one or more indices on the relation

- A transaction T_i that performs a lookup must lock all the index leaf nodes that it accesses, in S-mode

 - Even if the leaf node does not contain any tuple satisfying the index lookup (e.g. for a range query, no tuple in a leaf is in the range)

- A transaction T_i that inserts, updates or deletes a tuple t_i in a relation r

 - must update all indices to r

 - must obtain exclusive locks on all index leaf nodes affected by the insert/update/delete

- The rules of the two-phase locking protocol must be observed

Guarantees that phantom phenomenon won't occur

Next-Key Locking

Index-locking protocol to prevent phantoms required locking entire leaf

Can result in poor concurrency if there are many inserts

Alternative: for an index lookup

Lock all values that satisfy index lookup (match lookup value, or fall in lookup range)

Also lock next key value in index

Lock mode: S for lookups, X for insert/delete/update

Ensures that range queries will conflict with inserts/deletes/updates

Regardless of which happens first, as long as both are concurrent

Concurrency in Index Structures

Indices are unlike other database items in that their only job is to help in accessing data.

Index-structures are typically accessed very often, much more than other database items.

Treating index-structures like other database items, e.g. by 2-phase locking of index nodes can lead to low concurrency.

There are several index concurrency protocols where locks on internal nodes are released early, and not in a two-phase fashion.

It is acceptable to have nonserializable concurrent access to an index as long as the accuracy of the index is maintained.

In particular, the exact values read in an internal node of a B⁺-tree are irrelevant so long as we land up in the correct leaf node.

Concurrency in Index Structures (Cont.)

Example of index concurrency protocol:

Use **crabbing** instead of two-phase locking on the nodes of the B⁺-tree, as follows. During search/insertion/deletion:

- First lock the root node in shared mode.

- After locking all required children of a node in shared mode, release the lock on the node.

- During insertion/deletion, upgrade leaf node locks to exclusive mode.

- When splitting or coalescing requires changes to a parent, lock the parent in exclusive mode.

Above protocol can cause excessive deadlocks

- Searches coming down the tree deadlock with updates going up the tree

- Can abort and restart search, without affecting transaction

Better protocols are available; see Section 16.9 for one such protocol, the B-link tree protocol

- Intuition: release lock on parent before acquiring lock on child

- And deal with changes that may have happened between lock release and acquire

Weak Levels of Consistency

Degree-two consistency: differs from two-phase locking in that S-locks may be released at any time, and locks may be acquired at any time

- X-locks must be held till end of transaction

- Serializability is not guaranteed, programmer must ensure that no erroneous database state will occur]

Cursor stability:

- For reads, each tuple is locked, read, and lock is immediately released

- X-locks are held till end of transaction

- Special case of degree-two consistency

Weak Levels of Consistency in SQL

SQL allows non-serializable executions

Serializable: is the default

Repeatable read: allows only committed records to be read, and repeating a read should return the same value (so read locks should be retained)

However, the phantom phenomenon need not be prevented

T1 may see some records inserted by T2, but may not see others inserted by T2

Read committed: same as degree two consistency, but most systems implement it as cursor-stability

Read uncommitted: allows even uncommitted data to be read

In many database systems, read committed is the default consistency level

has to be explicitly changed to serializable when required

set isolation level serializable

Transactions across User Interaction

Many applications need transaction support across user interactions

- Can't use locking

- Don't want to reserve database connection per user

Application level concurrency control

- Each tuple has a version number

- Transaction notes version number when reading tuple

 - select** r.balance, r.version **into** :A, :version

 - from** r **where** acctId = 23

- When writing tuple, check that current version number is same as the version when tuple was read

 - update** r **set** r.balance = r.balance + :deposit

 - where** acctId = 23 **and** r.version = :version

Equivalent to **optimistic concurrency control without validating read set**

Used internally in Hibernate ORM system, and manually in many applications

Version numbering can also be used to support first committer wins check of snapshot isolation

- Unlike SI, reads are not guaranteed to be from a single snapshot

End of Module 16

Deadlocks

Consider the following two transactions:

T_1 : write (X)
 write(Y)

T_2 : write(Y)
 write(X)

Schedule with deadlock

T_1	T_2
lock-X on A write (A)	
	lock-X on B write (B) wait for lock-X on A
wait for lock-X on B	